

Lecture 11 MFML Sec9

Advanced Optimization

August 3, 2025

Contents

1	The Challenge of Non-Linear Optimization	2
1.1	The Hiker's Dilemma: An Analogy	2
1.2	The Enemies of Gradient Descent	2
1.2.1	Cliffs	2
1.2.2	Valleys (or Ravines)	2
2	Gaining Momentum: The Rolling Ball	2
2.1	The Intuition of Momentum	2
2.2	The Mathematics of Momentum (The Slide's Version)	2
3	Adaptive Learning Methods	3
3.1	AdaGrad: The Toll Booth	3
3.2	RMSProp: The Smart Toll Booth	3
3.3	Adam: The Ultimate Traffic System	4
4	Optimization with Rules: Constrained Problems	4
4.1	The Rule-Maker: Lagrange Multipliers	4
5	The Primal and The Dual: The Climber and the Ranger	5
5.1	The Primal Problem: The Climber Moves First	5
5.2	The Dual Problem: The Ranger Moves First	6
5.3	Weak vs. Strong Duality	6
6	The Beauty of Convexity	6
6.1	What is a Convex Problem?	6
6.2	Example: Why Negative Entropy is Convex	7
7	Solved Numerical Problems	7
7.1	Problem 1: Linear Programming Dual	7
7.1.1	Problem Statement	7
7.1.2	Solution	7
7.2	Problem 2: Quadratic Programming	8
7.2.1	Problem Statement	8
7.2.2	Solution	8

1 The Challenge of Non-Linear Optimization

1.1 The Hiker's Dilemma: An Analogy

Imagine you are a blindfolded hiker trying to find the lowest point in a national park. This is the core task of an optimization algorithm. Your best strategy is to feel the ground around you and take a step in the steepest downhill direction. This simple strategy is called **Gradient Descent**.

However, the terrain (our loss surface) isn't always a simple bowl. It can be treacherous, with two particularly nasty features: cliffs and valleys.

1.2 The Enemies of Gradient Descent

1.2.1 Cliffs

A cliff is a region where the slope of the terrain changes dramatically. On one side, the ground is almost flat, and on the other, it drops off sharply.

- **The Problem:** If our step size (learning rate) is too small, we will make agonizingly slow progress in the flat region. If our step size is large enough to make progress in the flat region, we will completely overshoot the bottom of the cliff, possibly landing on an even higher point on the other side. The algorithm struggles to find a learning rate that works for both the gentle and steep parts of the function.

1.2.2 Valleys (or Ravines)

A valley is a long, narrow depression in the terrain. The sides of the valley are very steep, but the floor of the valley slopes gently towards the lowest point.

- **The Problem:** The gradient will point steeply across the valley, not along its floor. As a result, our hiker (the algorithm) will take a large step and slam into the opposite wall. From there, the gradient will point back across the valley, and the algorithm will bounce from one side to the other, like a pinball. This frantic side-to-side oscillation makes for very slow progress along the valley towards the true minimum.

2 Gaining Momentum: The Rolling Ball

To solve the problem of valleys, we need a smarter way to move. Instead of a hiker who changes direction instantly, let's imagine a heavy ball rolling down the hill.

2.1 The Intuition of Momentum

A heavy ball has **momentum**. It builds up velocity in a consistent direction. It doesn't just consider the slope right under its feet; it also remembers the direction it was just moving. This "memory" is its inertia.

2.2 The Mathematics of Momentum (The Slide's Version)

The update rule from the slides beautifully captures this physical intuition:

$$x_{t+1} = x_t - \alpha \nabla f(x_t) + \beta(x_t - x_{t-1})$$

Let's break this down into its two main forces:

- **The Force of Gravity:** The term $-\alpha \nabla f(x_t)$ is the standard gradient descent step. This is like the force of “gravity,” always pulling the ball in the steepest downhill direction.
- **The Force of Inertia:** The term $+\beta(x_t - x_{t-1})$ is the momentum term. The vector $(x_t - x_{t-1})$ represents the previous step—the direction and speed the ball was just traveling. We multiply this by a friction parameter β (e.g., 0.9) and add it to our update. This is the ball’s inertia—its tendency to keep moving in the same direction.

How does this help?

- **In Valleys:** The “gravity” part (∇f) will point across the valley, causing oscillations. However, the “inertia” part will point mostly along the valley floor. Over several steps, the side-to-side gravity forces will tend to cancel each other out, while the inertia forces along the valley will accumulate, accelerating the ball towards the minimum.
- **On Flat Terrains:** The “gravity” part (∇f) will be tiny, threatening to stop our progress. But the “inertia” part ensures the ball keeps rolling based on its previous velocity, helping it glide across flat regions without slowing down too much.

3 Adaptive Learning Methods

Adaptive methods are even smarter. They realize that the ideal step size might be different for every single parameter. They give each parameter its own, personal learning rate that adapts over time.

3.1 AdaGrad: The Toll Booth

Analogy: The Toll Booth

Imagine each parameter is a road with a toll booth. Every time a gradient update passes through (a car drives by), the toll increases. If a road sees a lot of traffic (has large gradients), its toll becomes very high, making it expensive for future cars to pass. This slows down traffic on busy roads.

Simple Math Breakdown: For each parameter w_i , we maintain a cache, A_i , that accumulates the square of the gradients.

1. Calculate gradient: $g_{t,i} = \nabla f(w_{t,i})$
2. Update cache (increase toll): $A_{t,i} = A_{t-1,i} + g_{t,i}^2$
3. Update parameter (pay toll): $w_{t+1,i} = w_{t,i} - \frac{\alpha}{\sqrt{A_{t,i}} + \epsilon} g_{t,i}$

Downside: The toll only ever goes up. Eventually, all roads become so expensive that traffic grinds to a halt.

3.2 RMSProp: The Smart Toll Booth

Analogy: The Smart Toll Booth

This toll booth is smarter. It cares more about recent traffic than traffic from hours ago. The toll price is a mix of the old price and the price of the most recent car. If a road is quiet for a while, the toll slowly decreases.

Simple Math Breakdown: The only change is the cache update, which now uses a decaying average.

1. Calculate gradient: $g_{t,i} = \nabla f(w_{t,i})$
2. Update cache (smart toll): $A_{t,i} = \rho A_{t-1,i} + (1 - \rho)g_{t,i}^2$
3. Update parameter: $w_{t+1,i} = w_{t,i} - \frac{\alpha}{\sqrt{A_{t,i}} + \epsilon} g_{t,i}$

Here, ρ (e.g., 0.99) is a decay rate. This prevents the toll from growing forever.

3.3 Adam: The Ultimate Traffic System

Analogy: The Ultimate Traffic System

Adam is a complete traffic management system. It uses the “Smart Toll Booth” (RM-SProp) to control the speed of each car on each road. But it also has a “Traffic Director” (Momentum) who looks at the general flow of traffic and guides cars along the smoothest, most consistent routes.

Simple Math Breakdown: Adam uses two caches: m_t (the traffic director’s average direction) and v_t (the smart toll price).

1. Calculate gradient: $g_t = \nabla f(w_t)$
2. Update traffic director: $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$
3. Update smart toll: $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$
4. Update parameter: $w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} m_t$

Adam uses the direction from the traffic director (m_t) and scales it by the price from the smart toll ($\sqrt{v_t}$).

4 Optimization with Rules: Constrained Problems

4.1 The Rule-Maker: Lagrange Multipliers

Analogy: Personal Budgeting

Your goal is to minimize your spending, $f(x)$, where x represents the items you buy. However, you have a budget constraint: your spending cannot exceed your income. We can write this as $g(x) = \text{spending} - \text{income} \leq 0$. The Lagrange multiplier, λ , is the “price” or “pain” you feel for every dollar you go over budget.

The Lagrangian function combines these:

$$\mathcal{L}(x, \lambda) = \underbrace{f(x)}_{\text{Your Spending}} + \underbrace{\lambda g(x)}_{\text{Penalty for overspending}}$$

Key Intuition:

- If you are under budget ($g(x) < 0$), there is no pain. The best price for overspending is zero, so $\lambda = 0$.
- If you are over budget ($g(x) > 0$), the pain is immense. To make the problem work, the penalty λ becomes infinite to force you back within budget.

- If you are exactly on budget ($g(x) = 0$), the price λ has a real meaning: it tells you how much your objective (e.g., happiness from spending) would improve if your income increased by one dollar. It's the marginal value of your budget.

Visual Intuition: Imagine a topographic map of your spending function $f(x)$. The constraint $g(x) = 0$ is a fence drawn on this map. The optimal point is where the lowest possible contour line of your map just touches the fence. At this exact point, the gradient of the map ($\nabla f(x)$) and the gradient of the fence line ($\nabla g(x)$) will be perfectly parallel, pointing in opposite directions. The multiplier λ is the scaling factor that makes them equal in length: $\nabla f(x) = -\lambda \nabla g(x)$.

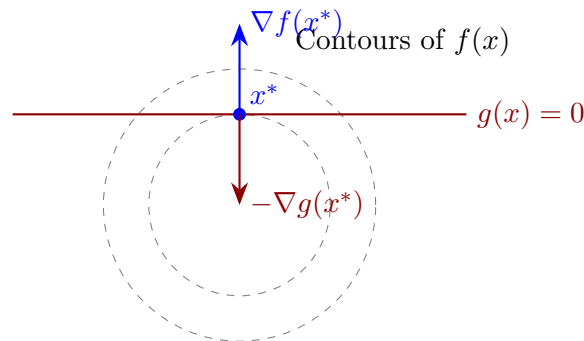


Figure 1: The gradient of the function (∇f) is parallel to the gradient of the constraint (∇g) at the optimal point. λ is the scaling factor.

5 The Primal and The Dual: The Climber and the Ranger

Analogy: The Mountain Climber and the Park Ranger

Imagine a **Climber** whose goal is to find the lowest point in a national park. The altitude of the park is given by the function $f(x)$. However, a **Park Ranger** has declared a certain area off-limits, defined by a fence where $g(x) \leq 0$. The Ranger will impose a fine for every meter the Climber strays past the fence.

The Lagrangian is the total "cost" to the climber:

$$\mathcal{L}(x, \lambda) = \underbrace{f(x)}_{\text{Altitude}} + \underbrace{\lambda g(x)}_{\text{Fine for trespassing}}$$

Here, $\lambda \geq 0$ is the fine per meter, set by the Ranger.

5.1 The Primal Problem: The Climber Moves First

The Climber must choose a location x , knowing the Ranger will then choose the fine λ to maximize the Climber's cost.

$$p^* = \min_x \underbrace{\left(\max_{\lambda \geq 0} \mathcal{L}(x, \lambda) \right)}_{\text{Ranger's maximum fine}}$$

The Game:

- If the Climber chooses a spot **outside** the fence ($g(x) > 0$), the Ranger can set an infinite fine ($\lambda = \infty$). The Climber's cost is infinite.

- If the Climber chooses a spot **inside** the fence ($g(x) \leq 0$), the term $\lambda g(x)$ is negative. To maximize the cost, the Ranger's best move is to set the fine to zero ($\lambda = 0$). The Climber's cost is just their altitude, $f(x)$.

The Climber's best strategy is to find the lowest altitude $f(x)$ while staying inside the fence. This is the original problem.

5.2 The Dual Problem: The Ranger Moves First

The Ranger sets the fine-per-meter, λ , first. Then, the Climber, seeing the fine, will find the location x that minimizes their total cost (altitude + fine).

$$d^* = \max_{\lambda \geq 0} \underbrace{\left(\min_x \mathcal{L}(x, \lambda) \right)}_{\text{Climber's best spot}}$$

The Game: The Ranger wants to set a fine λ that is as discouraging as possible. By setting the fine, the Ranger reshapes the landscape for the climber. The Ranger's goal is to choose a fine that makes the Climber's lowest possible point as high as possible. The Ranger is trying to establish the highest possible "floor" on the Climber's altitude.

5.3 Weak vs. Strong Duality

- **Weak Duality** ($d^* \leq p^*$): The highest floor the Ranger can set is always lower than or equal to the lowest point the Climber can actually reach. There might be a "duality gap" between them.
- **Strong Duality** ($d^* = p^*$): For "nice" problems (convex problems), the gap is zero. The Ranger's best floor is the same as the Climber's lowest point. The problem can be solved from either perspective.

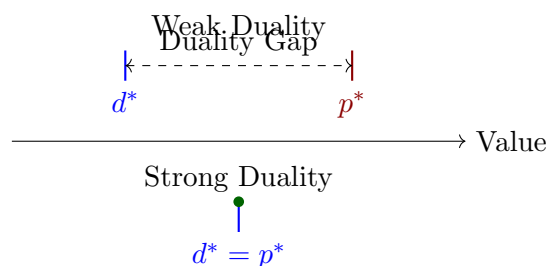


Figure 2: Weak Duality has a gap between the dual and primal solutions. Strong Duality closes this gap.

6 The Beauty of Convexity

6.1 What is a Convex Problem?

- **Convex Set:** A set where if you pick any two points and draw a straight line between them, the entire line segment stays within the set. A circle is convex; a star shape is not.
- **Convex Function:** A function where the line segment connecting any two points on its graph lies on or above the graph.

The Magic Property: A convex function has only one minimum: a global minimum. There are no local minima to get trapped in. This makes optimization much more reliable. A convex optimization problem is one that minimizes a convex function over a convex set.

6.2 Example: Why Negative Entropy is Convex

The negative entropy function is $f(x) = x \log(x)$ for $x > 0$. Let's prove it's convex using the second derivative test. A function is convex if its second derivative is non-negative.

1. First Derivative:

$$f'(x) = \frac{d}{dx}(x \log x) = 1 \cdot \log(x) + x \cdot \frac{1}{x} = \log(x) + 1$$

2. Second Derivative:

$$f''(x) = \frac{d}{dx}(\log x + 1) = \frac{1}{x}$$

For the domain of the function ($x > 0$), $f''(x) = 1/x$ is always positive. Therefore, the negative entropy function is strictly convex.

7 Solved Numerical Problems

7.1 Problem 1: Linear Programming Dual

7.1.1 Problem Statement

Consider the Linear Programming (LP) problem:

$$\begin{aligned} &\text{minimize} && x + 2y + 3z \\ &\text{subject to} && x + 3y + z \leq 1 \\ & && 2x + y \leq 2 \\ & && x + 3z \leq 3 \\ & && x, y, z \in \mathbb{R} \end{aligned}$$

- (a) Write down the Lagrangian dual of this problem.
- (b) State whether strong duality holds, and justify.

7.1.2 Solution

(a) Finding the Lagrangian Dual

Step 1: Write the Lagrangian. We have 3 inequality constraints, so we introduce 3 non-negative Lagrange multipliers: $\lambda_1, \lambda_2, \lambda_3 \geq 0$.

$$\begin{aligned} \mathcal{L}(x, y, z, \lambda) = & (x + 2y + 3z) + \lambda_1(x + 3y + z - 1) \\ & + \lambda_2(2x + y - 2) + \lambda_3(x + 3z - 3) \end{aligned}$$

Step 2: Find the Dual Function $D(\lambda) = \min_{x,y,z} \mathcal{L}$. To do this, we group the terms by our primal variables x, y, z :

$$\begin{aligned} \mathcal{L} = & x(1 + \lambda_1 + 2\lambda_2 + \lambda_3) \\ & + y(2 + 3\lambda_1 + \lambda_2) \\ & + z(3 + \lambda_1 + 3\lambda_3) \\ & - (\lambda_1 + 2\lambda_2 + 3\lambda_3) \end{aligned}$$

The Lagrangian is linear in x, y, z . For the minimum to be bounded (i.e., not $-\infty$), the coefficients of x, y, z must be zero.

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial x} &= 1 + \lambda_1 + 2\lambda_2 + \lambda_3 = 0 \\ \frac{\partial \mathcal{L}}{\partial y} &= 2 + 3\lambda_1 + \lambda_2 = 0 \\ \frac{\partial \mathcal{L}}{\partial z} &= 3 + \lambda_1 + 3\lambda_3 = 0\end{aligned}$$

If these conditions hold, the terms involving x, y, z vanish, and the minimum value of $\mathcal{L}$ is just the constant term.

$$D(\lambda) = -\lambda_1 - 2\lambda_2 - 3\lambda_3$$

Step 3: Write the Dual Problem. The dual problem is to maximize the dual function $D(\lambda)$ subject to the conditions we just found, plus the non-negativity of the multipliers.

$$\begin{aligned}\text{maximize} \quad & -\lambda_1 - 2\lambda_2 - 3\lambda_3 \\ \text{subject to} \quad & \lambda_1 + 2\lambda_2 + \lambda_3 = -1 \\ & 3\lambda_1 + \lambda_2 = -2 \\ & \lambda_1 + 3\lambda_3 = -3 \\ & \lambda_1, \lambda_2, \lambda_3 \geq 0\end{aligned}$$

(b) Strong Duality

Answer: Yes, strong duality holds.

Justification: The primal problem is a Linear Program (LP), because both the objective function and all constraint functions are linear (affine). For any LP, if the primal problem has an optimal solution (is feasible and bounded), then strong duality holds. This is a fundamental theorem of linear programming.

7.2 Problem 2: Quadratic Programming

7.2.1 Problem Statement

Solve the following Quadratic Program (QP):

$$\min_{x_1, x_2} \frac{1}{2}(x_1^2 + x_2^2) \quad \text{subject to} \quad x_1 + x_2 = 1$$

7.2.2 Solution

Step 1: Write the Lagrangian. This is an equality constraint, so we use a multiplier ν that is not restricted in sign.

$$\mathcal{L}(x_1, x_2, \nu) = \frac{1}{2}(x_1^2 + x_2^2) + \nu(x_1 + x_2 - 1)$$

Step 2: Minimize the Lagrangian. We find the partial derivatives with respect to x_1 and x_2 and set them to zero.

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial x_1} &= x_1 + \nu = 0 \implies x_1 = -\nu \\ \frac{\partial \mathcal{L}}{\partial x_2} &= x_2 + \nu = 0 \implies x_2 = -\nu\end{aligned}$$

Step 3: Use the constraint to solve for the multiplier. We substitute the results from Step 2 into the original constraint equation.

$$\begin{aligned}x_1 + x_2 &= 1 \\(-\nu) + (-\nu) &= 1 \\-2\nu &= 1 \\\nu &= -\frac{1}{2}\end{aligned}$$

Step 4: Find the optimal primal variables. Now that we have ν , we can find the optimal values for x_1 and x_2 .

$$\begin{aligned}x_1 = -\nu &= -\left(-\frac{1}{2}\right) = \frac{1}{2} \\x_2 = -\nu &= -\left(-\frac{1}{2}\right) = \frac{1}{2}\end{aligned}$$

Conclusion: The minimum value of the function occurs at $(x_1, x_2) = (\frac{1}{2}, \frac{1}{2})$. This problem is a convex optimization problem (a quadratic objective with a positive definite Hessian matrix and a linear constraint), so strong duality holds and this is the unique global minimum.